Forecasting Fundamentals

Study Notes: Time Series Data, Decomposition, Validation & Exponential Smoothing

---

1. What is Time Series Data?

A **time series** is a sequence of observations recorded in **chronological order**, typically at fixed, regular time intervals (hourly, daily, weekly, monthly, quarterly, yearly).

Formally: **y(t)**, for t = 1, 2, 3, ... T — a single variable observed repeatedly over time.

Examples

- Daily closing stock price of a company
- Monthly retail sales revenue
- Hourly website traffic / server load
- Quarterly GDP of a country
- Daily number of units sold per SKU (demand forecasting)

What makes it different from regular (cross-sectional) data?

Aspect	Cross-Sectional Data	Time Series Data
Order	Row order doesn't matter	Order is critical — time sequence carries information
Independence	Observations usually assumed independent	Observations are typically **correlated** with recent past (autocorrelation)
Splitting for validation	Random train/test split is fine	Must split by time (no random shuffling / no data leakage)
Goal	Explain relationships between variables	Often forecast future values using past values

---

2. Properties, Features & Assumptions of Time Series Data

2.1 Key Properties / Features

- **Temporal ordering** — each point has a fixed position in time; sequence cannot be shuffled
- **Autocorrelation** — current values are correlated with past values (self-correlation across lags)
- **Trend** — a long-term increase or decrease in the level of the series
- **Seasonality** — regular, calendar-related repeating patterns (daily, weekly, yearly cycles)
- **Cyclicity** — fluctuations that are not of fixed calendar frequency (e.g. business cycles, multi-year)

- **Noise / irregular component** — random variation left over after removing trend, seasonality, cycle
- **Heteroscedasticity** — variance of the series may change over time (common in financial series)

2.2 Core Assumption: Stationarity

Many classical forecasting models (ARIMA, and to some extent exponential smoothing on the residual structure) assume — or work better with — a **stationary** series. A series is (weakly/covariance) stationary if the following statistical properties do **not** change over time:

- **Constant mean** — no trend
- **Constant variance** — no changing spread (no heteroscedasticity)
- **Constant autocorrelation structure** — covariance between y(t) and y(t+k) depends only on the lag k, not on t itself

Note: *Most real-world business time series (sales, demand, revenue) are* ***non-stationary*** *— they have trend and/or seasonality. This is why differencing (for ARIMA) or explicit trend/seasonal modeling (ETS/Holt-Winters) exists — to transform or explicitly model away non-stationarity.*

2.3 How to test for stationarity (production checklist)

- **Visual check** — plot the series; look for obvious trend/seasonality/changing variance
- **ACF / PACF plots** — slow-decaying ACF suggests non-stationarity / trend
- **Statistical tests:**
  - Augmented Dickey-Fuller (ADF) test — null hypothesis: series has a unit root (non-stationary)
  - KPSS test — null hypothesis: series IS stationary (opposite null of ADF — often used together)
  - Rolling mean/variance plot — visually check if statistics shift over time windows

Note: *In production pipelines, ADF + KPSS are often run together because they have opposite null hypotheses — agreement between them gives more confidence than either alone.*

---

3. Components of Time Series Data

A time series is conceptually thought of as being generated by four underlying components:

Component	Symbol	Description	Example
Trend	T	Long-term overall direction (up/down/flat)	Steady growth in company revenue over 5 years
Seasonality	S	Fixed, calendar-based repeating pattern	Ice cream sales spike every summer
Cyclicity	C	Repeating pattern with no fixed period, driven by broader economic/business cycles	Recession-driven demand dips every 7-10 years
Residual / Irregular	R (or e)	Random, unpredictable noise remaining after removing T, S, C	One-off spike from a viral social media post

> ***Note:*** *Trend and Cycle are often combined into a single **'trend-cycle'** component in practice (hard to separate cleanly with limited history), commonly denoted simply as the **Trend** component.*

---

4. Decomposition of Time Series Data

Decomposition splits a series into its components (Trend, Seasonal, Residual) so each can be inspected, modeled, or removed independently. There are two standard mathematical forms:

4.1 Additive Decomposition

```
y(t) = T(t) + S(t) + R(t)
```

Use when the **magnitude of seasonal fluctuations is roughly constant** over time, regardless of the level of the series.

4.2 Multiplicative Decomposition

```
y(t) = T(t) × S(t) × R(t)
```

Use when seasonal fluctuations **scale with the level** of the series — e.g., as a retail business grows, December's seasonal spike grows proportionally larger too.

> ***Note:*** *Quick production heuristic: plot the series. If seasonal swings look like a constant band → additive. If the swings visibly widen as the series grows → multiplicative. Alternatively, log-transform the series and apply additive decomposition — mathematically equivalent to multiplicative on the original scale.*

4.3 Common Decomposition Methods

Method	Notes
Classical decomposition	Simple moving-average based method; oldest approach; struggles at series edges and with evolving seasonality
STL (Seasonal-Trend decomposition using LOESS)	Robust, flexible; allows seasonal component to change over time; handles outliers well; widely used in production (R: stl(), Python: statsmodels STL)
X-11 / X-13ARIMA-SEATS	Used heavily by government statistical agencies (e.g., census data); very robust for official economic series

4.4 Why decomposition matters in production

- **Diagnostics** — quickly spot whether trend/seasonality exist before choosing a model family
- **Feature engineering** — deseasonalized data can be fed into simpler models, or seasonal indices used as features for ML models
- **Anomaly detection** — large residuals after decomposition flag outliers/anomalies
- **Model selection signal** — strength of trend/seasonality (measurable via variance of residual vs. detrended/deseasonalized series) helps decide between SES, Holt, Holt-Winters, ARIMA, etc.

---

5. Validation Set & Performance Metrics

5.1 Why time series validation is different

You **cannot** randomly shuffle and split time series data the way you would for standard ML (e.g. k-fold CV) — this leaks future information into the training set and produces unrealistically good, misleading performance estimates.

5.2 Correct splitting strategies

a) Simple Train / Test (Hold-out) Split

Train on the earlier portion, test on the most recent portion in chronological order. Simple but only gives you a single point estimate of accuracy — risky for high-variance series.

b) Rolling-Origin Evaluation (a.k.a. Time Series Cross-Validation / Walk-Forward Validation)

The gold standard in production. The training window progressively expands (or slides) forward in time, and you forecast the next step(s) at each origin, then compare against actuals.

- **Expanding window:** training set grows each iteration, always starting from t=1
- **Sliding window:** training set is a fixed size, moves forward each iteration (better when older data becomes irrelevant, e.g. after a structural break)

> ***Note:*** *This mimics how the model will actually be used in production: retrained periodically, forecasting forward only using data that would have been available at that point in time.*

5.3 Common Performance Metrics

Metric	Formula (conceptual)	Notes
MAE (Mean Absolute Error)	avg(|actual – forecast|)	Same units as data; robust to outliers; easy to explain to stakeholders
MSE / RMSE	avg((actual – forecast)^2), then sqrt for RMSE	Penalizes large errors heavily; RMSE is in original units, MSE is not
MAPE (Mean Absolute Percentage Error)	avg(|actual – forecast| / |actual|) × 100	Scale-independent, easy to communicate as %; breaks down / undefined when actuals are near zero (common with intermittent demand)
sMAPE (Symmetric MAPE)	Normalizes by (|actual| + |forecast|)/2	Attempts to fix MAPE's asymmetry and zero-division issue, but still has known quirks
MASE (Mean Absolute Scaled Error)	MAE of model / MAE of naive seasonal forecast	Scale-independent AND well-behaved near zero; compares against a naive baseline; preferred in competitions (e.g. M4)

WAPE / WMAPE (Weighted MAPE)	sum(|actual – forecast|) / sum(|actual|)	Common in retail/demand forecasting across many SKUs — aggregates fairly without being skewed by low-volume items

5.4 Production guidance on metric choice

- Avoid plain **MAPE** as the sole metric if the series can be zero or near-zero (intermittent demand, new products)
- **MASE** or **WAPE** are generally safer defaults for cross-series comparison (e.g. across thousands of SKUs of different scales)
- Always compare against a **naive baseline** (naive, seasonal-naive) — a 'sophisticated' model that can't beat naive isn't adding value
- Report a **distribution** of errors across the rolling-origin folds, not just a single average — stakeholders should know worst-case as well as average-case performance
- For business decisions (inventory, staffing), also evaluate **prediction interval coverage** (e.g., does the actual fall inside the 80% interval ~80% of the time?), not just point-forecast accuracy

---

6. Exponential Smoothing Models

6.1 Core idea

Forecast = a weighted average of past observations, where weights **decay exponentially (geometrically)** the further back in time you go — recent data matters more than old data.

6.2 Simple Exponential Smoothing (SES)

Handles a series with **level only** — no trend, no seasonality. Forecast is always **flat** for all future horizons.

```
Level(t) = α · y(t) + (1 – α) · Level(t–1)
Forecast(t+1) = Level(t)
```

- α ($0 < \alpha < 1$): smoothing parameter, fitted via minimizing SSE / MLE, not chosen manually
- High $\alpha \rightarrow$ reacts fast to recent changes (short memory, ~1/α effective periods)
- Low $\alpha \rightarrow$ very smooth, slow to react (long memory)
- Mathematically equivalent to ARIMA(0,1,1)

6.3 Holt's Linear Method (Double Exponential Smoothing)

Extends SES to handle **level + trend**. Two smoothing parameters: α (level) and β (trend).

```
Level(t) = α·y(t) + (1–α)·(Level(t–1) + Trend(t–1))
Trend(t) = β·(Level(t) – Level(t–1)) + (1–β)·Trend(t–1)
Forecast(t+h) = Level(t) + h · Trend(t)
```

> ***Note:*** *Plain linear trend extrapolated far out can produce unrealistic forecasts (unbounded growth/decline). In production, prefer the* ***damped trend*** *variant, which flattens the trend as the horizon grows (adds a damping*

parameter $\phi < 1$).

6.4 Holt-Winters Method (Triple Exponential Smoothing)

Extends Holt's method to handle **level + trend + seasonality**. Three smoothing parameters: α (level), β (trend), γ (seasonality).

- **Additive Holt-Winters** — use when seasonal swings are roughly constant in absolute size
- **Multiplicative Holt-Winters** — use when seasonal swings scale with the level of the series

6.5 ETS Framework (modern production formulation)

ETS = **E**rror, **T**rend, **S**eason — a state-space statistical reformulation of the entire exponential smoothing family. Each component can be None (N), Additive (A), or Multiplicative (M), and trend can additionally be damped (Ad).

ETS notation	Equivalent to
ETS(A,N,N)	Simple Exponential Smoothing
ETS(A,A,N)	Holt's Linear Method
ETS(A,Ad,N)	Damped Trend method
ETS(A,A,A)	Additive Holt-Winters
ETS(A,A,M)	Multiplicative Holt-Winters

- Provides a proper likelihood function → AIC/BIC for automatic model selection, and valid prediction intervals
- Libraries: Python — statsmodels (ExponentialSmoothing), sktime / Nixtla statsforecast (AutoETS); R — forecast::ets()

6.6 Production Considerations

- **Great baseline / production-scale workhorse** — fast, robust, no manual tuning needed; scales to thousands/millions of series (e.g. Nixtla statsforecast fits ETS on massive panels quickly via vectorization)
- **No native support for exogenous variables** (promotions, price, weather, holidays) — this is the most common reason production teams move to ARIMAX, regression-based, or ML approaches
- **Cold-start problem** — needs a reasonable amount of history to estimate level/trend/seasonal reliably; new SKUs/series need a fallback strategy
- **Not designed for intermittent demand** (many zeros) — use Croston's method or similar instead
- **Always use damped trend** in production for any series where trend is extrapolated more than a few periods ahead
- **Refit periodically** — parameters (α, β, γ) should be re-estimated on a schedule as new data arrives, not left static indefinitely

- Often used as **one candidate among several** (ETS, ARIMA, Prophet, LightGBM) in an ensemble or per-series model selection pipeline, chosen via backtested accuracy (Section 5)

End of notes.